



National Autonomous University of Mexico

Faculty of Engineering

Division of Electrical Engineering



Computer Engineering

Computer Graphics and Human-Computer Interaction

Final Project 2

Semester: 2026-1

Theoric Group: 05

320118597

Deadline: 24/11/2025

Introduction

In this technical manual I explain how I made my final project “Pokémon Center”, which I made in C++ with OpenGL 3.3. The main idea was to create a 3D scene with simple realistic lights, simple textures, simple shaders, and simple keyboard animations.

The project also has a laboratory with glass windows made with a glass shader, and a second room with five extra objects. Each object was modeled, textured, and added to the same OpenGL pipeline.

During the semester I made many small practices, and this final project is the mix of all of them: 3D models, shaders, blending, first-person camera, animations, skybox, point lights, and special materials like glass and emissive textures.





Project Purpose

The purpose of this project was to use everything I learned in the semester to create a full 3D scene in C++ and OpenGL. The idea was to build a Pokémon Center and then add a laboratory and a second room with five more objects, using 3D models, lights, textures, shaders, and animations.

With this project I could show that I can load models, use custom shaders, use blending, move the camera, add lights, and organize a big OpenGL project.

Objectives

General Objective

To create an interactive 3D scene in OpenGL where I can use models, lights, shaders, textures, and animations, showing the final result of what I learned.

Specific Objectives

- Add many 3D models: the Pokémon Center, the laboratory, and the second room with five more objects.
- Use custom shaders: light shader, glass shader for windows, and emissive shader.
- Set the lights of the scene with directional and point lights.
- Use a skybox to give environment to the scene.
- Use transparency and blending, especially on the laboratory windows (V2).
- Add a first-person camera to move around the scene.
- Add simple and complex animations, like Pokéballs that jump and spin in the air.
- Make the windows have different textures and different transparency.
- Explain the whole process in this technical manual.

Project Scope

The project includes everything to make a complete 3D scene in OpenGL. I added many 3D models, lights, shaders, camera, blending, and animations. I also added the laboratory with its new windows (V2) and a second room with five extra objects.

The project uses .obj models, textures with transparency, custom shaders (glass, light, emissive), skybox, directional lights, point lights, and a first-person camera. I also set the render order so transparent and solid objects look correct.

Facades to Model

Pokémon Center Reference

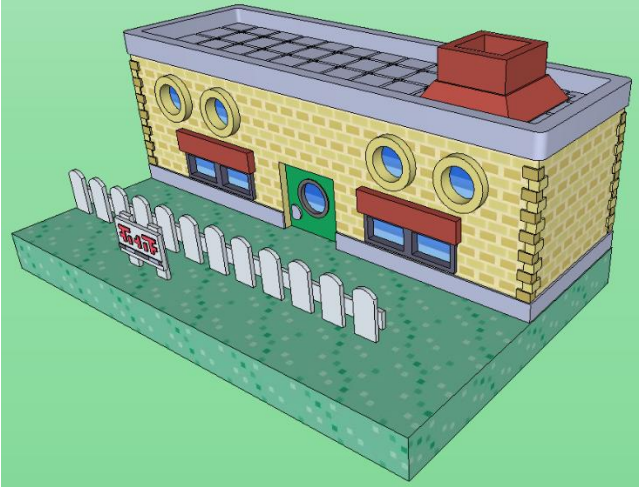


Final Result

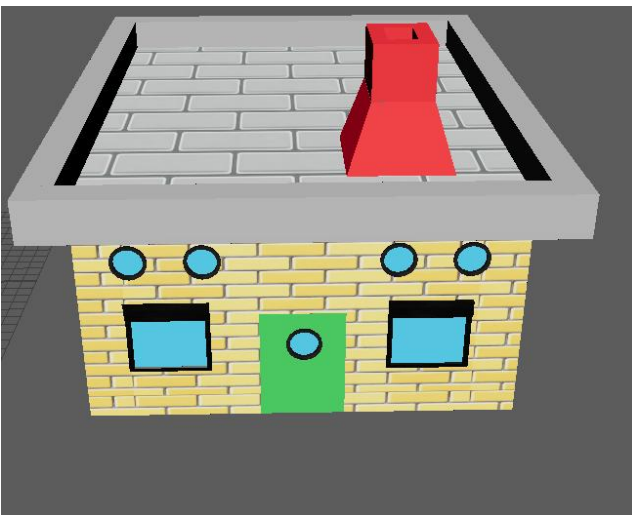


3853 vertices and 3228 faces

Laboratory

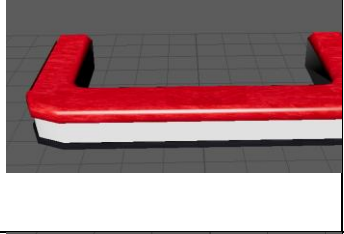
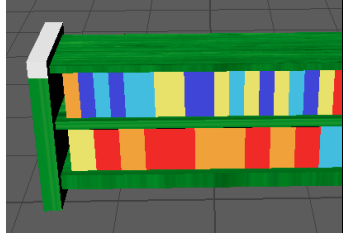
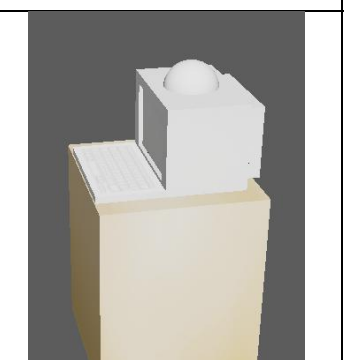


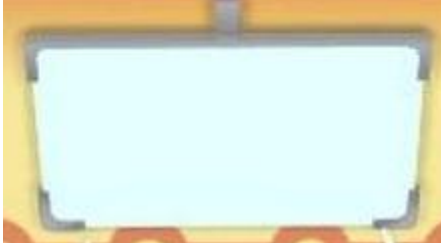
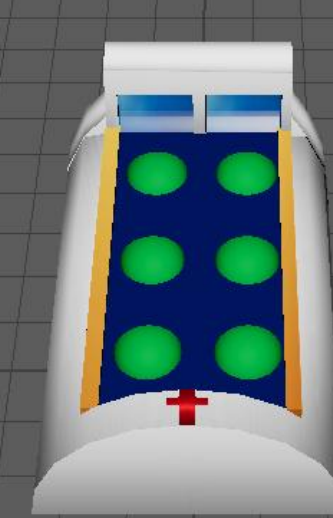
Final Result

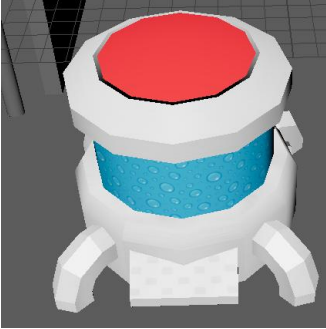
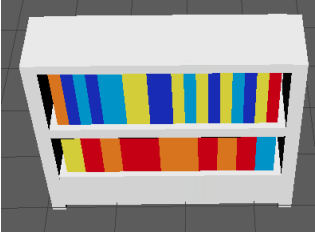


1220 vertices and 2329 faces

Objects to Model

Referencia	Resultado final	Datos
		354 vertices and 410 faces
		286 vertices and 298 faces
		12 vertices and 8 faces
		498 vertices and 948 faces

		394 vertices and 216 faces Model: https://sketchfab.com/3d-models/pokemon-center-scene-c497ebb33_ca343a6a4cc4ab685b9235c Creator: kiri-rxrai
		244 vertices and 388 faces
		1480 vertices and 2857 faces
		64 vertices and 100 faces

		242 vertices and 424 faces Modelo: https://sketchfab.com/models/fbe85c2f81a04063aa573f9161824734/embed Creador: QuangCao
		40 vertices and 60 faces
		127 vertices and 125 faces
		3437 vertices and 3284 faces

Herramientas utilizadas

Tool	Function
Maya	I made all objects here using only basic shapes. When I needed more complex shapes, I changed them, cut them, or joined them. I also made the textures here. I used square images with power of 2, and in the UV Editor I mapped the textures.
Gimp	This free image editor let me change colors, change images to RGB, and make the image square with power of 2. It helped me fix textures fast.
Navegador de internet	On internet pages I searched and found references and objects that I could use again for my scene. I also took some textures for my models from there.
Visual Studio	Here I loaded all models, the facade, and I joined them. Here I also created shaders that helped me make the animations of my objects. I also moved, rotated, and scaled the models here, including backgrounds and lights.

Animations Evidence

Here are the animations programmed in my Pokémon Center project. Each one is controlled by keys in the final program.

Animation 1: Voltorb Movement

Description:

Voltorb moves left and right between two limits. It also makes a small “bounce” when it reaches the edges.

The movement uses linear interpolation controlled by time (`glfwGetTime`) and a sine function to make the acceleration smooth.

```
if (anim1_on) {  
    posX += dir * speed * deltaTime;
```

```

if (posX > maxX || posX < minX) dir = -dir;
model = glm::translate(glm::mat4(1.0f), glm::vec3(posX, yBase, zPos));
}

```



Animation 2: Pokéball Sequence

Technical Description:

The Pokéballs jump one by one in a parabolic jump. Each one activates after the other.

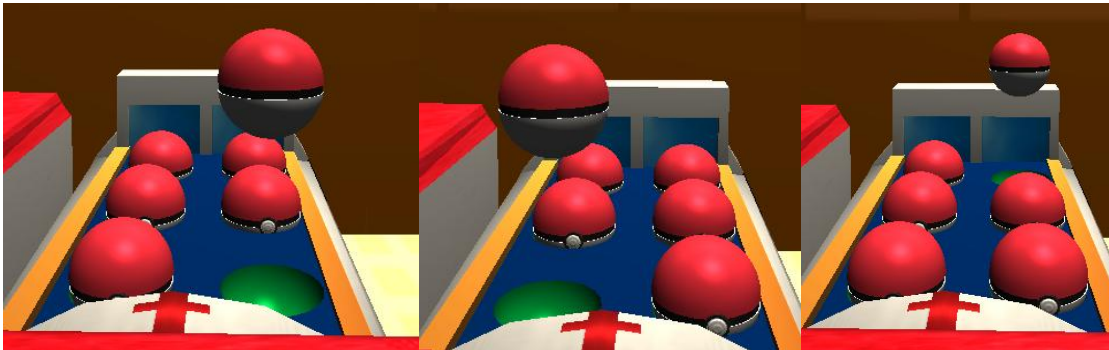
The vertical movement uses a quadratic equation of gravity:

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$$

```

if (anim2_on) {
    bolas[i].y = y0 + v0 * t - 0.5f * 9.8f * t * t;
    if (bolas[i].y <= y0) nextBall();
}

```



Animation 3: TV (CRT Shader)

Technical Description:

The TV uses a shader that combines: texture horizontal lines (scanlines) small flicker made with $\sin(uTime * frequency)$ This gives the TV an old CRT effect.

```
float flicker = 0.8 + 0.2 * sin(uTime * 12.0);
float scan = 0.9 + 0.1 * sin(vUV.y * 100.0);
FragColor = texture(uTex, vUV) * flicker * scan;
```





Animation 4: Pulsing Light

Description:

A simulated radial light effect made using a sine wave in intensity (uMin → uMax).

It uses smoothstep to soften the light radius.

Use mask = $1.0 - \text{smoothstep}(r0, r0 + u\text{Soft}, r)$;



Animation 5: PC Screen (animPC Shader)

Description:

The computer monitor uses a “digital interference” animation. The texture shows small temporal variations.

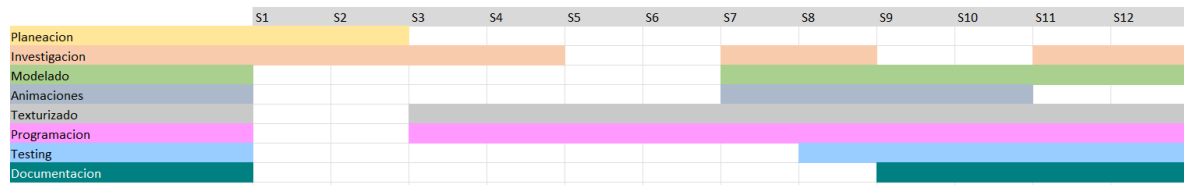
It uses `uTime` to move noise patterns and change brightness.

```
vec2 offset = vec2(sin(uTime*3.0)*0.01, cos(uTime*4.0)*0.01);  
vec3 color = texture(uTex, vUV + offset).rgb;  
FragColor = vec4(color, 1.0);
```



Gantt

Notes



I took a long time to start the first facade and the first room in OpenGL because of errors and mistakes in the system, and texture colors that were not in RGB mode. Maybe also because the size of the images was not power of 2 textures.

For modeling, I always modeled one object per week or worked on the facade, and I always finished the object with textures. The Pokéball object was reused because it was originally planned for the second room.

The second house facade was easier because I already had the experience from the first facade.

Animations were also delayed because I didn't know how to use the objects until the shader animation practice.

Making animations with shaders was easier than doing animations for each object, which took more time.

Limitations

During this project I had many technical and personal problems. This was my first big OpenGL project, and I had to learn to organize my time better between modeling, programming, textures, lights, and testing.

One of the biggest limitations was my lack of experience with shaders.

At the beginning, I had trouble understanding the light shader, the glass shader, and the emissive shader, especially when I had to make them work correctly with textures and blending. This took me more time than I expected.

Another limitation was planning. Sometimes I spent too much time on small details like texture position or a light angle, and this made me late with bigger tasks. I had to reorganize my schedule many times.

I also had technical problems with file paths, .obj loading, GitHub Desktop, and the final executable. Textures often didn't load, appeared black, or the model showed without materials.

Even with these problems, each obstacle helped me understand better how a full OpenGL project works, and I was able to complete all models, shaders, lights, and animations in one final scene.

Project Value Estimation

In total, this project took me around 15 to 20 real work hours, including modeling, texturing, and implementation in OpenGL.

To estimate the value, I used a price of 10 to 20 USD per hour, which is the normal price for a beginner 3D artist or an intermediate technical developer.

Approximate Hour Distribution

Dividing the hours into the main project tasks, the time is something like:

- Modeling and geometry of buildings and objects:

About 6 to 8 hours

(creation of the Pokémon Center facade, laboratory, and interior objects)

- Texturing and UVs in GIMP/Maya:

About 3 to 4 hours

(creating and fixing textures, UV maps, stretch corrections)

- Technical implementation in C++ and OpenGL:

About 4 to 5 hours

(loading models, shaders, lights, camera, skybox, blending)

- Testing, fixing and final adjustments:

About 2 to 3 hours

(fixing lighting, transparency, positions, and small errors)

If I use an intermediate price of 15 USD/hour, the cost is:

Modeling: ~90–120 USD

Texturing and UVs: ~45–60 USD

Technical work: ~60–75 USD

Testing and fixes: ~30–45 USD

Total estimated project value: 250–270 USD

Final Price Range

Using the full price range (10 to 20 USD per hour), the total value is between:

150 USD (15 h × 10 USD/h)

400 USD (20 h × 20 USD/h)

So, for my experience level and the time I worked, a fair price would be between 250 and 270 USD with an intermediate rate.

Why I chose Maya

I chose Autodesk Maya as my main modeling tool because it is a professional and stable program that makes it easier to create clean and detailed 3D models.

During the project, Maya helped me organize object hierarchies, adjust pivots, add materials, and make geometric changes without losing control of the model.

Its UV mapping tools and polygon optimization were very useful to prepare the models before exporting them in .obj format, making sure everything worked correctly in OpenGL.

The visual interface and preview tools helped me understand better the lighting and proportions of the scene, which was very important to create a more realistic environment.

In summary, Maya gave me a smooth workflow between artistic modeling and the technical part of rendering, helping connect 3D design with programming in C++ and OpenGL.

Why I chose GIMP

I chose GIMP because it is a free and complete tool for creating and editing textures.

It helped me adjust colors, transparency, and visual details quickly.

Working with layers made it easier to edit textures for my models.

It also helped me export images in formats compatible with OpenGL, like .png, keeping good quality so they looked correct in the project.

Conclusion

The development of the Pokémon Center and the Laboratory in OpenGL was a complete process that combined graphic programming, 3D modeling, shaders, textures, and animations inside a real-time rendering environment.

In the project, I created a full visual system with seven light sources, dynamic effects using GLSL shaders, and independent animations for different scene elements (Voltorb, Pokéballs, TV, PC monitor, light bulb, and windows).

Each part was designed and modified using base codes from previous practices and adapting them to a much bigger and unified project.

The experience helped me improve my skills in C++ and OpenGL, using libraries like GLFW, GLEW, GLM, and SOIL2, and also integrating external models from Blender or Sketchfab. I also fixed common rendering problems like black textures (non-POT textures), animation timing issues, and transparency management.

I was also able to create a working executable, including .dll dependencies and folders, so the project could run outside the IDE.

Writing this document and the user manual helped me learn the importance of explaining how a technical system works in a clear way.

In conclusion, this project not only improved my technical knowledge of rendering and animation, it also helped me in creativity, planning, and understanding how to connect multiple parts of a graphic computing project into one functional, interactive scene.

References

Photos Public Domain. (2012, 24 mayo). *Bright red paper texture* [Fotografía]. <https://www.photos-public-domain.com/2012/05/24/bright-red-paper-texture/>

Wong, A. (s. f.). *Pintura de pared blanca con línea negra* [Fotografía]. Unsplash. <https://unsplash.com/es/fotos/pintura-de-pared-blanca-con-linea-negra-3Om4DHcaAc0>

<https://sketchfab.com/models/fbe85c2f81a04063a-a573f916182-4734/embed>

QuangCao. Proffesor Oak Laboratory. Sketchfab. <https://sketchfab.com/models/fbe85c2f81a04063a-a573f916182-4734/embed>

Bulbagarden Archives. (s. f.). Pokédex Entry ZA (Pikachu) [Imagen]. https://archives.bulbagarden.net/wiki/File:Pok%C3%A9dex_Entry_ZA.png

Jeff. (s. f.). *Pokémon Center Scene* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/pokemon-center-scene-c497ebb33ca343a6a4cc4ab685b9235c>

GoblinCore. (s. f.). *Voltorb – Daily Program* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/daily-program-9228a3c2a3a24d87a47b5edb25c474e2>

Facultad de Ingeniería, UNAM. (s. f.). *Laboratorio de Computación Gráfica e Interacción Humano Computadora* [Sitio web institucional]. <https://www.ingenieria.unam.mx/>